POCKETWISE

Student Budgeting & Expense Management

Take control of your money. One decision at a time.

UX/UI DESIGN BOOK

A mobile-first financial management concept for university students in Taiwan.

PROJECT DETAILS	VALUE
Project type	Individual academic design project
Primary audience	University students in Taiwan
Currency convention	New Taiwan dollar (NTD / NT$)
Deliverables	Design documentation, presentation PDF and clickable HTML/CSS/JavaScript prototype
Prepared	2 November 2026
Status	Concept design – assumptions awaiting user validation

Prepared as an academic design concept. This document distinguishes proposed design decisions from research findings that still require validation.

STUDENT: Phume | STUDENT ID: 411416640 | DEPARTMENT: GFLEP

Executive summary

PocketWise is a proposed mobile-first budgeting and expense-tracking experience designed for university students in Taiwan. It gives students a clear view of money coming in, money going out, category-level spending, budgets and savings goals. The initial concept uses New Taiwan dollars (NT$) and everyday student expense examples such as meals, transport, study materials, mobile data and entertainment.

The design focuses on a common and testable challenge: when students manage money across different everyday purchases, it can be difficult to maintain a reliable picture of current spending and how much remains available.

PocketWise responds with quick manual transaction entry, visible budget progress and a simple dashboard.

This is a design-only academic project supported by a clickable front-end prototype. It is not a live banking product and does not connect to financial institutions. No interviews, surveys or usability tests are claimed as completed; research questions, personas and success targets are proposed for validation.

Project at a glance

Question	Design answer
Why is it needed?	To make everyday student money management more visible, structured and less dependent on memory.
Who is the primary user?	A university student in Taiwan who manages a limited monthly allowance, income or savings.
What task matters most?	Record a purchase and understand its effect on the remaining budget.
Where is friction expected?	Repeated manual recording, unclear categories, forgotten purchases and difficulty interpreting remaining budget.
What is the MVP?	Dashboard, manual income/expense entry, categories, budgets, savings goals and basic insights.
What is excluded?	Bank linking, payments, transfers, credit, investment advice and automated financial predictions.

Document map

The book covers project overview, problem context, users, goals, experience, information architecture, user flows, wireframes, visual design, technical plan, management and AI collaboration. The accompanying presentation summarises the same design story; the prototype demonstrates key front-end interactions.

1. Project overview

Project name: PocketWise

Category: FinTech concept / UX/UI design

Project format: Individual academic project

Target setting: University students in Taiwan **Currency:** New Taiwan dollar (NT$ / TWD)

Service purpose

PocketWise helps students organise personal finances by recording income and expenses, setting category budgets, checking available funds and tracking savings goals. It aims to support awareness and planning rather than provide regulated financial services.

Project scope

In scope for the design MVP	Out of scope for the initial MVP
Mobile-first dashboard and navigation	Live bank-account aggregation

Manual income and expense entry	Money transfers and payment processing
Transaction history and categories	Credit, lending or investment products
Monthly category budgets and progress	Automated bank feeds and receipt OCR
Savings goals and progress indicators	AI financial advice or predictive scoring
Basic insights and responsive layouts	Native app release and production authentication

Project constraints

The deadline is short and the deliverable is design-focused. The prototype therefore uses sample data and browser-local storage; it does not represent a secure production financial system. Research access to Taiwanese university students is not confirmed, so user assumptions must be validated before implementation.

Team and roles

The project is completed by one student. The student is responsible for project direction, research planning, UX, UI, prototyping, documentation and presentation. AI assistance is used as a drafting and coding aid and is disclosed in Section 12.

2. Problem context

Problem statement

University students may receive money from family support, scholarships, part-time work or savings and use it across frequent daily expenses. When purchases are not recorded consistently, students may lose visibility of spending patterns and find it harder to judge whether their remaining funds will cover upcoming needs. PocketWise proposes a lightweight system for recording transactions and seeing budget impact.

Context of use

Context	Design implication
Between classes / on the move	Fast, thumb-friendly transaction entry; short forms and clear feedback.
At a shop or café	Quick amount and category entry; no assumption of reliable receipt capture.
Weekly or monthly planning	Readable totals, budget progress and time-period filters.
Low attention / small screen	Prioritise the balance and key actions; avoid dense tables.
Variable connectivity	Show clear save/error states; prototype data remains in the browser.

Known limitations

1. Manual entry creates friction and can be forgotten.
2. A category may not fit every purchase; users need an editable or uncategorised option.
3. Sample budgets cannot reflect every student's living costs.
4. The design cannot verify balances against a bank account.
5. Currency and language preferences may vary; NTD is the initial default, with localisation planned.
6. A front-end prototype is not suitable for storing real sensitive financial information.

Research inputs and evidence status

Inputs currently available: the assignment brief, the chosen concept, the target context (Taiwanese university students) and the selected currency (NTD). No primary research evidence has yet been collected. The following are hypotheses, not findings: students want faster expense entry; students benefit from seeing remaining category budgets; and users prefer plain-language financial summaries over complex analytics.

Validation plan

Conduct 5–8 short student interviews or a small survey; ask participants how they currently track spending, what causes them to lose track, which categories matter, and what information they need before making a purchase. Avoid collecting

account numbers, bank credentials or exact sensitive financial records. Record anonymised themes and revise the design accordingly.

3. Users

Primary user

A university student in Taiwan who manages a limited monthly allowance, part-time income, scholarship funds or personal savings and wants a simple way to monitor everyday spending.

Related roles

Role	Need / responsibility
Primary student user	Record transactions, check budgets and monitor savings.
Secondary: student mentor / peer	May share budgeting tips; no access to another user's private records.
Future support administrator	Could manage help content and categories in a production service; not part of the prototype MVP.

Proto-persona

Attribute	Illustrative persona: Mei-Ling
Profile	Undergraduate student in Taiwan; lives away from family during term (assumption to validate).
Financial pattern	Receives a monthly allowance and occasional part-time income; makes frequent small purchases.
Goals	Know how much remains, avoid overspending and save for a personal goal.
Behaviours	Checks phone frequently; may rely on memory or notes to track purchases.
Pain points	Forgets small purchases, finds manual spreadsheets cumbersome and is unsure which category is consuming the budget.
Needs	Quick add-expense action, simple categories, clear remaining budget and reassuring feedback.
Quote (illustrative)	“I want to know where my money went without doing a lot of calculations.”

Assumptions requiring validation

- Students are comfortable entering expenses manually.
- A monthly budget is a useful planning unit for the target group.
- Users understand category budgets and progress bars.
- NT$ is the preferred display currency for the selected Taiwan audience.
- Students value savings goals alongside expense tracking.
- English-first labels are acceptable for the initial academic prototype; Traditional Chinese localisation may be required for real deployment.

Accessibility and inclusion

Use readable type sizes, sufficient colour contrast, clear labels in addition to colour, large touch targets, plain-language errors and a layout that remains usable with screen magnification. Never to assume all users have the same financial literacy or income pattern.

4. Goals and success criteria

Design problem statement

How might we help university students in Taiwan understand and manage everyday spending with less effort, so they can make more confident decisions about their available money?

Product goals

1. Make current money status visible at a glance.
2. Make a common transaction-entry task quick and understandable.
3. Show budget progress in context, not just isolated totals.
4. Encourage achievable savings goals without judgement.
5. Communicate errors and recovery options clearly.

Proposed success criteria

Measure	Proposed target	How to evaluate
Task completion	At least 90% complete add-expense task in a moderated test	Observe 5–8 target users using a prototype.
Task time	Median add-expense task under 60 seconds	Time from opening form to visible confirmation.
Comprehension	At least 4/5 users correctly identify remaining monthly budget	Ask users to interpret dashboard.
Navigation	At least 80% find budgets and transaction history without help	First-click or short navigation test.
Error recovery	At least 4/5 users recover from invalid amount / failed save	Scenario-based prototype test.

These are proposed targets, not measured outcomes. They should be adjusted after baseline testing and actual user feedback.

5. Experience strategy

Primary task

The key task is: record a purchase and understand how it changes the user's available balance and category budget.

User journey

Stage	User action / thought	Touchpoint	Friction	Opportunity
1. Need arises	“Can I afford this?”	Dashboard	Balance may be outdated or unclear.	Show available balance and remaining budget.
2. Purchase made	Wants to remember the expense.	Quick-add action	Manual entry can be forgotten.	Make add action prominent and short.
3. Record purchase	Enters amount and category.	Add transaction form	Unclear category or invalid amount.	Use suggested categories and inline validation.
4. Review impact	Checks spending progress.	Dashboard / budgets	Totals lack context.	Update category progress and remaining amount.

5. Plan ahead	Sets aside money for a goal.	Savings page	Goal may feel abstract.	Show target, saved amount and next milestone.

Experience priorities

Priority 1: quick, reliable expense entry. **Priority 2:** clear available balance and category budget. **Priority 3:** understandable history and savings progress. **Priority 4:** optional insights and personalisation.

Design opportunities

- Place “Add expense” in a persistent, easy-to-reach location.
- Default date to today but allow editing.
- Show currency formatting consistently as NT$ with whole-dollar examples.
- Give immediate confirmation after saving.
- Explain what happens when a category budget is exceeded.
- Allow correction or deletion of a mistaken transaction with confirmation.

6. Structure: content inventory and information architecture

Content inventory

Area	Content / data	Primary action
Dashboard	Available balance, income, expenses, budget summary, recent transactions	Add transaction; view details
Transactions	Date, title, amount, type, category, note	Search, filter, edit, delete
Budgets	Period, category, planned limit, spent, remaining	Create / edit budget
Savings goals	Goal name, target, saved amount, target date	Create goal; add contribution
Insights	Period totals, category breakdown, simple trend	Change period; inspect category
Profile / settings	Display name, currency, language, privacy/help links	Edit preferences

Navigation model

Use a five-item bottom navigation on mobile: **Home**, **Activity**, a prominent central **+** action, **Budgets** and **Goals**. Insights and settings are accessible from the dashboard/profile area.

Sitemap

Level 1	Level 2	Level 3 / detail
Home	Dashboard	Balance; month summary; budget snapshot; recent activity
Activity	Transactions	All; income; expenses; transaction detail
Add transaction	New entry	Expense / income; amount; category; date; note; confirmation
Budgets	Budget overview	Category budget; progress; edit budget

Goals	Savings overview	Goal detail; contribution; progress
Insights	Spending overview	Period selector; category breakdown
Profile	Preferences	Currency; language; privacy; help

Navigation labels

Usage of familiar action labels: “Add expense”, “Add income”, “View all”, “Edit budget”, “Create goal”, “Add to goal”, “Save transaction” and “Try again”. Avoid jargon such as “liquidity”, “asset allocation” or “cash-flow optimisation”.

7. User flows

Main flow: record an expense

Step	Screen / decision	System response
1	Open dashboard	Show current balance and quick actions.
2	Tap Add (+)	Open transaction type selector / form.
3	Choose Expense	Show amount, category, date and optional note.
4	Enter amount and category	Validate amount > 0; require a category.
5	Tap Save expense	Persist prototype record locally; update totals.
6	Confirmation	Show saved message and updated balance / budget.

Main flow diagram

Dashboard → Add (+) → Expense form → Validate fields → Save → Confirmation → Updated dashboard

Error / recovery path: invalid amount or failed save

Trigger	Feedback	Recovery
Amount is blank, zero or negative	Inline message: “Enter an amount greater than NT$0.”	Keep entered values; focus amount field; retry.
No category selected	Inline message: “Choose a category to continue.”	Keep amount and note; select category.
Browser storage unavailable / save fails	“We couldn't save this transaction. Your details are still on this screen.”	Offer Retry; do not clear the form.
User deletes by mistake	Confirmation dialog before deletion	Cancel to keep transaction; confirm to remove.

.

8. Layout: major-page wireframes

The following low-fidelity layouts describe the screen hierarchy and primary controls. The clickable prototype includes a higher-fidelity visual treatment of the same core pages.

A. Dashboard

■ PocketWise Profile ■
■ Good afternoon, Mei-Ling ■
■ ■
■ AVAILABLE BALANCE ■
■ NT$ 12,480 ■
■ Income NT$ 18,000 Expenses NT$ 5,520 ■
■ ■
■ [+ Add expense] [+ Add income] ■
■ ■
■ MONTHLY BUDGET ■
■ Food NT$ 2,100 / 4,000 ■
■ Transport NT$ 650 / 1,200 ■
■ Study NT$ 480 / 1,500 ■
■ ■
■ RECENT ACTIVITY ■
■ Lunch Food -NT$ 120 ■
■ Metro Transport -NT$ 40 ■
■ ■
■ Home Activity + Budgets Goals ■
■■■

■ ■
■ Note (optional) ■
■ [e.g. lunch with classmates] ■
■ ■
■ [Save expense] ■
■■■

10. Technical plan

MVP definition

A front-end concept prototype demonstrating the core student budgeting tasks. It uses sample transactions and browser localStorage to preserve demo state between reloads on the same browser. It is not production-ready and must not be used to store real financial records.

Proposed technology

Layer	MVP / prototype choice	Rationale
Interface	HTML5	Semantic structure and quick delivery.
Styling	CSS3	Responsive layout, design tokens and component styling.
Interaction	Vanilla JavaScript	No framework setup; transparent, easy-to-review logic.
Demo data	JavaScript seed data + localStorage	Allows add/edit/delete interactions without a backend.
Future backend	Node.js API + PostgreSQL (proposal only)	Could support accounts, durable records and server-side validation.
Future hosting	GitHub Pages for static prototype; production hosting to be assessed	Simple shareable demo; no sensitive data or secrets.

Data source and data model

The prototype uses fictional sample data created for demonstration; it does not import external or bank data.

Suggested entities: User (id, displayName, currency, language); Transaction (id, type, amount, category, date, note); Budget (id, period, category, limit); Goal (id, title, targetAmount, savedAmount, targetDate). Store amounts as integer NT dollars in the demo to avoid floating-point currency errors.

Privacy and security considerations

A real product would require secure authentication, server-side authorisation, encryption in transit and at rest, data minimisation, secure session management, backup and deletion processes, and a privacy review appropriate to Taiwan's legal requirements. Do not store bank credentials. The browser prototype is for fictional demo data only.

GitHub plan

Repository: pocketwise-student-budgeting. Suggested structure: /prototype/index.html, /prototype/styles.css, /prototype/app.js, /docs/design-book.pdf, /docs/presentation.pdf, README.md. Use commits such as “Add dashboard layout”, “Implement transaction validation” and “Document MVP scope”. Keep secrets out of the repository. Use issues or a simple project board for tasks.

Preliminary deployment plan

1. Review and test the static prototype locally.
2. Push source files to a GitHub repository.
3. Enable GitHub Pages for the prototype folder or deploy the static site through an approved static host.
4. Test mobile and desktop breakpoints and browser storage behaviour.
5. Share a demo link with a clear notice that all data is fictional and local to the browser.
6. Do not connect real financial accounts or collect real personal financial data.

Risks and mitigations

Risk	Mitigation
Manual input fatigue	Keep the form short; evaluate quick-add and defaults.
Users misunderstand remaining balance	Define balance labels and test comprehension.
Data loss in browser demo	Show save feedback; provide reset demo data; clarify local-only storage.
Scope creep	Protect MVP boundary; defer integrations.
Language mismatch	Validate English vs Traditional Chinese needs; design for localisation.

11. Project management

Task allocation

Workstream	Owner	Output
Project definition and scope	Student	Project brief and MVP boundary
Research planning	Student	Interview questions, assumptions and persona
Experience and IA	Student	Journey, content inventory, sitemap and flows
Wireframes and visual design	Student	Major page layouts and design system
Prototype implementation	Student with AI coding assistance	HTML/CSS/JS interactive prototype
Documentation	Student with AI drafting assistance	Design Book and presentation
Quality assurance	Student	Requirements audit, visual review and test notes

Schedule and milestones

Date / timebox	Milestone	Completion evidence
2 Nov – project setup	Scope and content structure locked	Approved project brief and requirements checklist
2 Nov – design production	Personas, IA, journey, flows and wireframes drafted	Sections 2–8 complete
2 Nov – visual / prototype	Design system and prototype assembled	Responsive screens and working interactions
3 Nov – finalisation	QA, PDF export and submission package	Design Book PDF, presentation PDF, source ZIP
3 Nov – final review	Check filenames, opening, page order and links	Final submission checklist

Milestone acceptance checklist

- All 12 assignment requirements are visibly covered.
- Research assumptions are not presented as findings.
- NTD is used consistently.
- Sitemap, flows and wireframes agree with the feature scope.
- Prototype includes main path and error/recovery path.
- MVP exclusions are explicit.
- PDFs open correctly and source files are included.

12. AI collaboration record

AI use statement

AI tools were used as a collaborative aid during the development of this academic design project. Their use included brainstorming the product concept, structuring the design-book sections, drafting initial UX content, proposing user flows and wireframe structures, generating front-end code and checking the submission against the assignment requirements.

Human responsibility

The student remains responsible for the project direction, selecting and reviewing design decisions, checking factual claims, adapting the content to course expectations, testing the prototype and ensuring the final submission is accurate. AI-generated assumptions are labelled and must not be represented as actual research evidence.

Transparency and limitations

No primary user interviews or usability tests are claimed as completed in this version. Any personas are proto-personas, not research-validated representations. The prototype uses fictional data and does not provide financial advice or connect to bank services. The student should disclose AI use in accordance with the institution's academic integrity policy and retain records of prompts and revisions if required.

AI collaboration log (summary)

Activity	AI contribution	Student review / action
Ideation	Suggested student budgeting concept and feature options	Selected PocketWise and narrowed the MVP.
Documentation	Drafted section structure and initial wording	Review, correct and align with assignment brief.
UX design	Proposed persona, journey, sitemap and flows	Treat as hypotheses; validate with target users.
Prototype code	Generated HTML, CSS and JavaScript structure	Run locally, test interactions and revise.
QA	Mapped deliverables to the 12 required areas	Final human review before submission.

Conclusion

PocketWise proposes a focused way for university students in Taiwan to record everyday spending, understand category budgets and track savings goals. Its design strategy prioritises quick entry, clear financial summaries and supportive feedback. The initial scope is deliberately limited to a design concept and front-end prototype; real-world release would require user research, accessibility testing, security engineering, privacy review and validation with students.

Appendix: prototype contents

The accompanying source package contains index.html, styles.css, app.js and a README. Open index.html in a modern browser to explore the sample dashboard, transaction management, budgets and savings goals. Data is stored locally in the browser for demonstration only.

POCKETWISE

Implementation, validation and delivery addendum

Student: Phume **Student ID:** 411416640 **Department:** GFLEP

This addendum extends the existing Design Book. The original concept, user experience, information architecture, wireframes and visual design remain unchanged. It adds practical implementation guidance and makes the status of technical evidence explicit.

13. Working prototype and technical implementation

A responsive front-end prototype has been prepared using HTML5, CSS3 and vanilla JavaScript. It demonstrates the core budgeting loop using fictional sample data: dashboard summaries, manual income and expense entry, category validation, transaction filtering and deletion, category budget progress, savings-goal creation and contributions, and browser-local persistence.

Data source: The prototype uses sample data and browser localStorage. It does not retrieve bank data or connect to a financial institution. Amounts are stored as whole New Taiwan dollars for the demonstration.

Data entities: Transaction (id, type, amount, category, date, note); Budget (category, limit); Goal (id, name, target, saved).

14. Google Apps Script and Google Sheets integration

A lightweight Google Apps Script connector is included in the source package. It accepts the current prototype data and writes it to three Google Sheets tabs: Transactions, Budgets and Goals. The connector is prepared code, not a live connection. It becomes an implemented integration only after the student creates a Sheet, adds the script, deploys the web app, configures its URL in the prototype and verifies a successful sync.

15. Git, GitHub and preliminary deployment

Repository name: **pocketwise-student-budgeting**. Suggested structure: prototype/index.html, prototype/styles.css, prototype/app.js, google-apps-script/Code.gs, docs/ and README.md. Use small, descriptive commits such as “Add responsive dashboard”, “Implement transaction validation” and “Add Sheets connector”. Keep credentials and personal data out of the repository.

The static front-end can be published through GitHub Pages after local testing. The Apps Script service is deployed separately. Record the repository URL, commit history, deployment settings and live Pages URL only after those steps have actually been completed. A deployment plan is not evidence of a completed deployment.

16. Testing and DevTools evidence

T01	Open dashboard at desktop/mobile sizes	No layout breakage; cards reflow at mobile width	To test
T02	Add valid expense	Transaction appears; balance and budget update	To test
T03	Add income	Income and available balance update	To test
T04	Blank/zero/negative amount	Inline error; no record saved	To test

T05	No category selected	Clear category error; entered data retained	To test
T06	Filter / delete transaction	Filter works; deletion requires confirmation	To test
T07	Reload browser	Demo state persists in same browser	To test
T08	Create goal / add contribution	Goal and saved amount update	To test
T09	DevTools Console and keyboard focus	No uncaught errors; visible focus and usable controls	To test
T10	Google Sheets sync	Three tabs populated after deployment	Not configured

17. Research and evaluation plan

The proto-persona and stated friction points are hypotheses. No primary research or usability testing is claimed as completed in the current documentation. If access permits, ill conduct 5–8 short interviews or a small survey with university students in Taiwan. Ask how they currently track spending, what causes missed records, which categories they use and what information they need before making a purchase.

Then run 5–8 moderated usability sessions (or a clearly labelled pilot if fewer participants are available). Ask participants to add an expense, find a transaction, interpret remaining budget and recover from an invalid amount. Record sample size, participant profile at a non-identifying level, task completion, time, errors and design changes. Do not invent findings or present AI-generated persona details as participant evidence.

18. Objective alignment and evidence map

Course objective	PocketWise evidence / remaining action
1. User-centred UI/UX	Problem statement, target user, proto-persona, assumptions and proposed success criteria. Validate with students.
2. Journey, IA, sitemap, wireframes	Existing Design Book sections 5–8; maintain consistency with prototype navigation.
3. Semantic and accessible HTML/CSS	Prototype uses semantic sections, labels, table structure, responsive CSS and focus styling. Run keyboard and contrast checks.
4. JavaScript interaction / async	Prototype includes forms, events, validation, rendering and localStorage. Sheets sync uses fetch; test after endpoint deployment.
5. DevTools and debugging	Use Console, Application/Storage and responsive device toolbar; add dated screenshots and a short issue/fix log.
6. Responsible AI use	AI collaboration record included; student must review code, understand it and disclose use per institutional rules.
7. Apps Script / Sheets	Code connector prepared. Deployment, URL configuration and successful sync still need to be completed and evidenced.
8. Git / GitHub	Repository and commit plan documented. Create repository, commit files and capture history.
9. Working solution and presentation	Prototype source included; test it, document design/technical choices and include actual test/deployment evidence in final review.

19. AI collaboration and human responsibility

AI tools supported ideation, document structuring, UX drafting, proposed user flows and wireframe content, front-end code generation and refinement, preparation of a Google Apps Script example, and an assignment-requirements audit. The student selected the concept, set the scope, reviews and adapts outputs, remains responsible for understanding and explaining the work, and must verify functionality and technical claims. Personas and research insights remain assumptions until validated.